Système de Gestion de Pharmacie

Projet réalisé par : Youssef Jbeniani / Ihsen Tlich

Introduction:

Ce projet vise à concevoir une application de gestion d'une pharmacie permettant :

- **l'authentification des utilisateurs** (Pharmacien / Gestionnaire) via login et mot de passe,
- **la gestion des médicaments** : ajout, suppression, modification du prix, et mise à jour du stock,
- **l'enregistrement et l'annulation des ventes**, avec mise à jour automatique du stock,
- **la consultation des ventes** (par médicament, par client et par période),
- **la gestion des commandes de réapprovisionnement** et la consultation des commandes,
- **le suivi du stock critique** (liste des médicaments dont le stock est inférieur à un seuil),
- **le suivi de l'historique du stock** (traces des entrées/sorties via StockHistorique),

Modèle conceptuel des données (MCD / MERISE)

1.1 Entités et attributs

PHARMACIEN

- **id_pharmacien** (identifiant)
- nom
- prenom
- login (unique)
- pwd

GESTIONNAIRE

- **id_gestionnaire** (identifiant)
- nom

- prenom
- login (unique)
- pwd

CLIENT

- **id_client** (identifiant)
- nom
- prenom
- email
- adresse

MEDICAMENT

- **id_medicament** (identifiant)
- nom
- dosage
- stock
- prix_unitaire

VENTE

- **id_vente** (identifiant)
- date
- quantite

COMMANDE

- **id_commande** (identifiant)
- date
- quantite

STOCK_HISTORIQUE

- **id_stock_hist** (identifiant)
- date
- quantite (delta + / -)

Associations et cardinalités (d'après les FK du schéma)

Pharmacien — Vente

- Un Pharmacien réalise **0..N** ventes
- Une Vente est enregistrée par **1** Pharmacien
  ➡️ `PHARMACIEN (1) ── (0..N) VENTE`

Client — Vente

- Un Client réalise **0..N** ventes

- Une Vente concerne **1** Client
  ➡️ `CLIENT (1) ── (0..N) VENTE`

Medicament — Vente

- Un Medicament apparaît dans **0..N** ventes
- Une Vente concerne **1** Medicament
  ➡️ `MEDICAMENT (1) ── (0..N) VENTE`

Gestionnaire — Commande

- Un Gestionnaire crée **0..N** commandes
- Une Commande est créée par **1** Gestionnaire
  ➡️ `GESTIONNAIRE (1) ── (0..N) COMMANDE`
- **Medicament — Commande**
- Un Medicament apparaît dans **0..N** commandes
- Une Commande concerne **1** Medicament
  ➡️ `MEDICAMENT (1) ── (0..N) COMMANDE`

Medicament — StockHistorique

- Un Medicament possède **0..N** lignes d'historique
- Une ligne d'historique concerne **1** Medicament
  ➡️ `MEDICAMENT (1) ── (0..N) STOCK_HISTORIQUE`

Diagramme de classe :

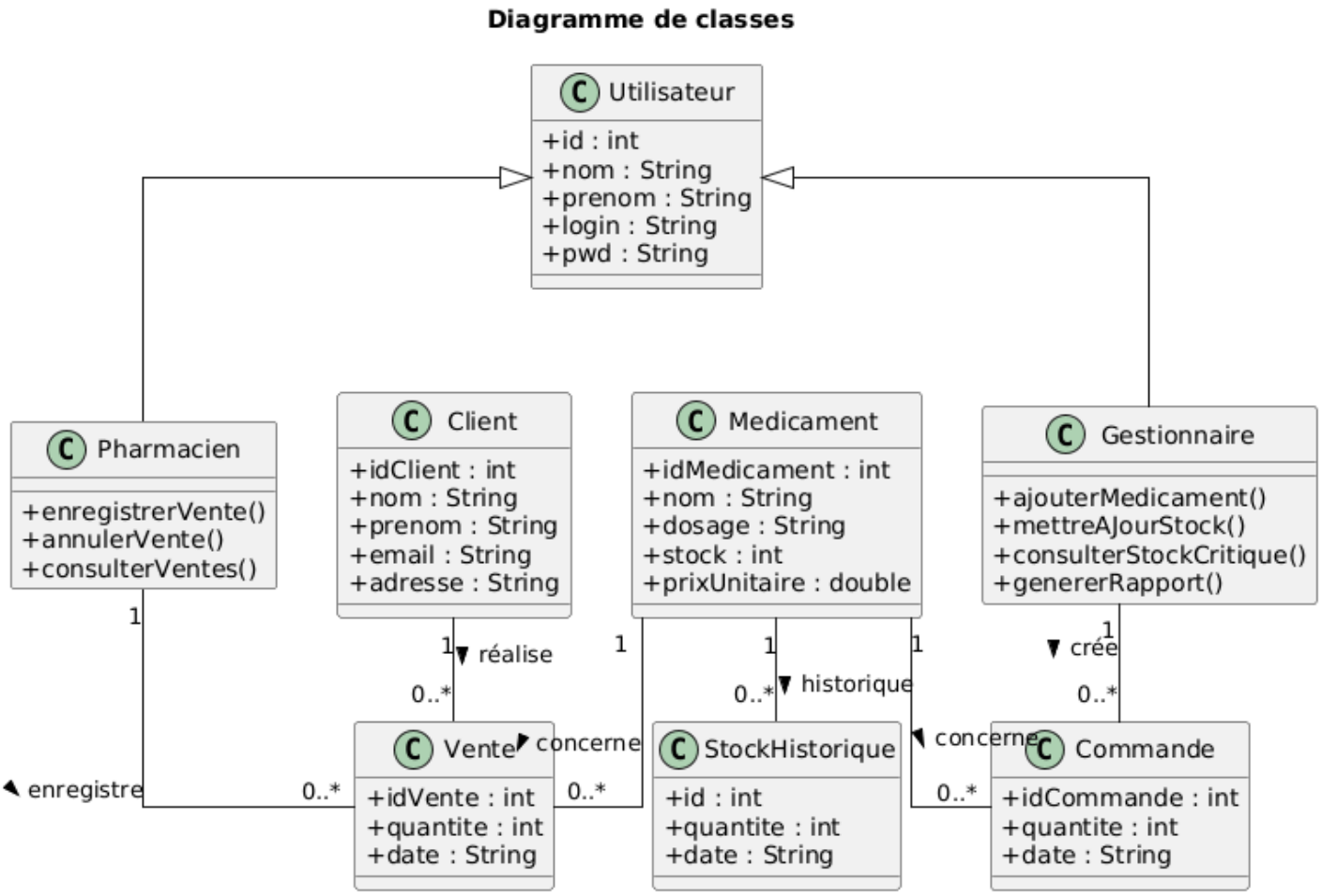
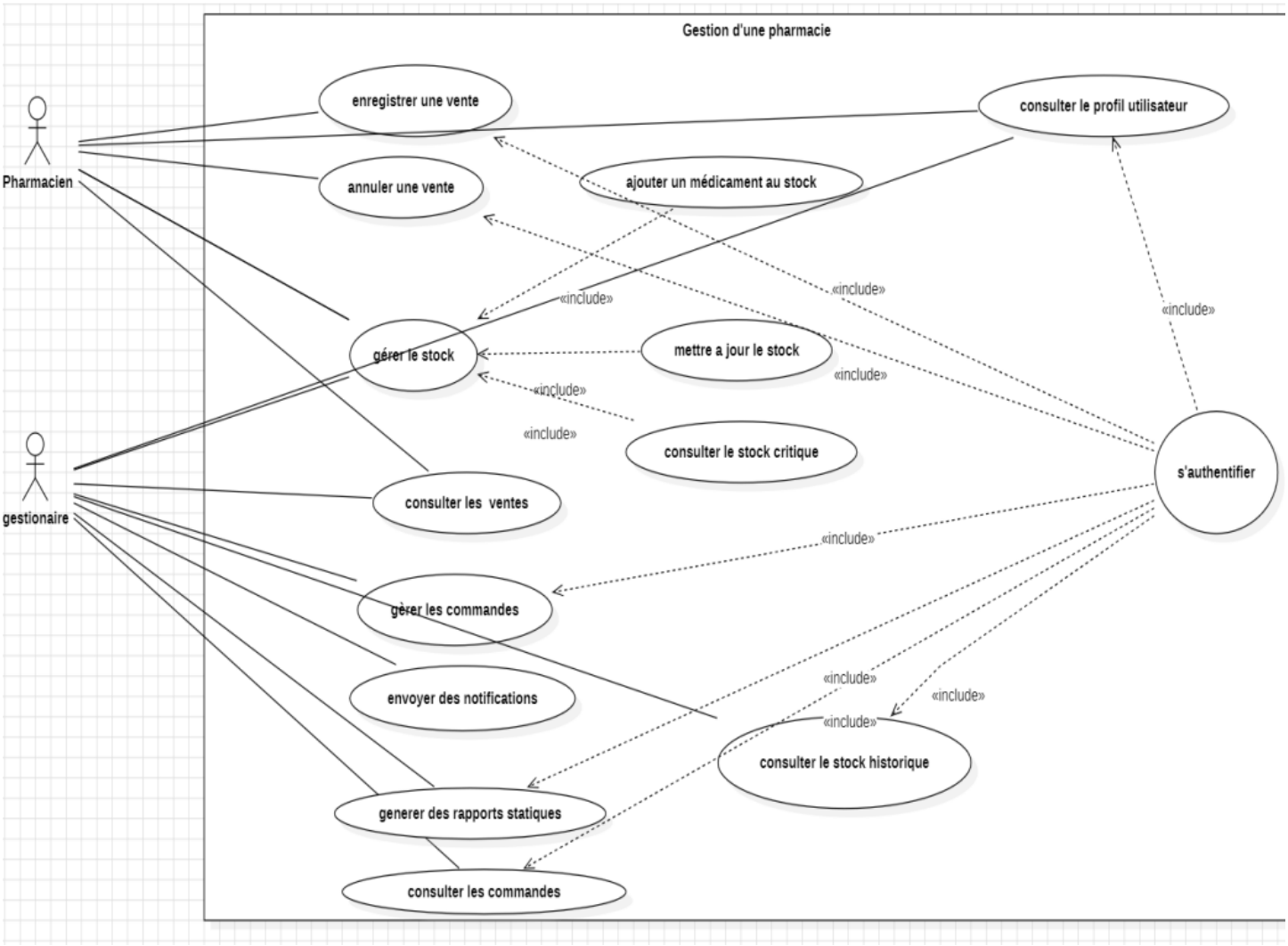


Diagramme de cas/utilisation:



Base de donnees :

Les tables:

- Tables (8)
  - Client
  - Commande
  - Gestionnaire
  - Medicament
  - Pharmacien
  - StockHistorique
  - Vente
  - sqlite_sequence

Exemples de contenue des tables :

	id_medicament	nom	dosage	stock	prix_unitaire
	Filter	Filter	Filter	Filter	Filter
1	2	Grippex	150	12	8.0
2	3	Panadol	200	14	13.0
3	6	Zinox	200	0	8.0

	id_client	nom	prenom	email	adresse
	Filter	Filter	Filter	Filter	Filter
1	1	Youssef	Jbeniani	coldlasagna7@gmal.com	tunis
2	2	Tlich	Ihsen	ihsentlich123@gmail.com	tunis

	id_gestionnaire	nom	prenom	login	pwd
	Filter	Filter	Filter	Filter	Filter
1	1	Doe	Gestion	gest	gest123

	id_pharmacien	nom	prenom	login	pwd
	Filter	Filter	Filter	Filter	Filter
1	1	Doe	Pharma	pharma	pharma123

Les classes du code :

Client:

```java
public class Client {

    private int idClient;
    private String nom;
    private String prenom;
    private String email;
    private String adresse;


    public Client(int idClient, String nom, String prenom, String email, String adresse) {
        this.idClient = idClient;
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.adresse = adresse;
    }
```

Gestionnaire:

```java
public class Gestionnaire {

    private int idGestionnaire;
    private String nom;
    private String prenom;
    private String login;
    private String pwd;

    public Gestionnaire(int idGestionnaire, String nom, String prenom, String login, String pwd) {
        this.idGestionnaire = idGestionnaire;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }
```

Pharmacien:

```java
public class Pharmacien {

    private int idPharmacien;
    private String nom;
    private String prenom;
    private String login;
    private String pwd;



    public Pharmacien(int idPharmacien, String nom, String prenom, String login, String pwd) {
        this.idPharmacien = idPharmacien;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }
```

Commande:

```java
public class Commande {

    private int idCommande;
    private int idGestionnaire;
    private int idMedicament;
    private int quantite;
    private String date; // ISO string

    public Commande(int idCommande, int idGestionnaire, int idMedicament, int quantite, String date) {
        this.idCommande = idCommande;
        this.idGestionnaire = idGestionnaire;
        this.idMedicament = idMedicament;
        this.quantite = quantite;
        this.date = date;
    }
```

Medicament:

```java
public class Medicament {

    private int idMedicament;
    private String nom;
    private String dosage;
    private int stock;
    private double prixUnitaire;

    public Medicament(int idMedicament, String nom, String dosage, int stock, double prixUnitaire) {
        this.idMedicament = idMedicament;
        this.nom = nom;
        this.dosage = dosage;
        this.stock = stock;
        this.prixUnitaire = prixUnitaire;
    }
```

Session:

```java
public class Session {

    private String role;

    private int userId;
    private String nom;
    private String prenom;

    public Session(String role, int userId, String nom, String prenom) {
        this.role = role;
        this.userId = userId;
        this.nom = nom;
        this.prenom = prenom;
    }
```

Vente:

```java
public class Vente {

    private int idVente;
    private int idPharmacien;
    private int idClient;
    private int idMedicament;
    private int quantite;
    private String date;


    public Vente(int idVente, int idPharmacien, int idClient, int idMedicament, int quantite, String date) {
        this.idVente = idVente;
        this.idPharmacien = idPharmacien;
        this.idClient = idClient;
        this.idMedicament = idMedicament;
        this.quantite = quantite;
        this.date = date;
```

L’application:

```java
public class App {

    // Scanner unique pour toute l'application (lecture console).

    private static final Scanner sc = new Scanner(System.in);

    // Méthode principale : boucle infinie de login -> menu -> logout.

    public static void main(String[] args) {
        System.out.println("=== PHARMACIE APP (JDBC + SQLite) ===");

        // Service d'authentification (appelle AuthDao -> SQLite)
        AuthService authService = new AuthService();

        while (true) {
            System.out.println();
            System.out.println("----- LOGIN -----");

            // Saisie login / mot de passe
            String login = readLine("Login: ");
            String pwd = readLine("Password: ");

            // Tentative de connexion
            Session session;
            try {
                session = authService.login(login, pwd);
            } catch (Exception e) {
                // Erreur technique (ex: driver, base introuvable, SQL error)
                System.out.println(" Erreur login: " + e.getMessage());
                continue;
            }

            // Si session == null => login/pwd incorrects
            if (session == null) {
                System.out.println(" Login incorrect.");
                continue;
            }

            System.out.println(" Connecté: " + session.getRole() + " | "
                    + session.getPrenom() + " " + session.getNom());

            // Redirection selon le rôle
            String role = session.getRole() == null ? "" : session.getRole().toLowerCase();
            if (role.contains("pharmac")) {
                menuPharmacien(session);
            } else if (role.contains("gestion")) {
                menuGestionnaire(session);
            } else {
                System.out.println("⚠ Rôle inconnu: " + session.getRole());
            }

            // Retour ici après logout
            System.out.println(" Déconnecté.");
        }
    }
```

```java
    // ============================================================
    //                      MENU PHARMACIEN
    // ============================================================

    private static void menuPharmacien(Session session) {
        // Service métier Pharmacien (utilise DAOs en interne)
        PharmacienService service = new PharmacienService(session);

        while (true) {
            System.out.println();
            System.out.println("===== MENU PHARMACIEN =====");
            System.out.println("1) Enregistrer une vente");
            System.out.println("2) Annuler une vente");
            System.out.println("3) Consulter ventes par médicament");
            System.out.println("4) Consulter ventes par client");
            System.out.println("5) Consulter ventes par période");
            System.out.println("6) Créer un client");
            System.out.println("0) Logout");

            int choix = readInt("Choix: ");

            switch (choix) {

                case 1 -> {
                    // Enregistrer vente : insert Vente + update stock (-qte) + log historique
                    try {
                        int idClient = readInt("ID client: ");
                        int idMed = readInt("ID medicament: ");
                        int qte = readInt("Quantité: ");

                        int idVente = service.enregistrerVente(idClient, idMed, qte);

                        System.out.println("☑ Vente enregistrée. ID Vente = " + idVente);
                        // NOTE : l'alerte "stock bas" est gérée dans PharmacienService
                    } catch (Exception e) {
                        printError(e);
                    }
                }

                case 2 -> {
                    // Annuler vente : delete Vente + update stock (+qte) + log historique
                    try {
                        int idVente = readInt("ID vente à annuler: ");
                        service.annulerVente(idVente);
                        System.out.println("☑ Vente annulée.");
                    } catch (Exception e) {
                        printError(e);
                    }
                }
```

```
case 3 -> {
    // Consulter ventes par médicament
    try {
        int idMed = readInt("ID medicament: ");
        List<VenteDao.Vente> ventes = service.ventesParMedicament(idMed);

        if (ventes.isEmpty()) {
            System.out.println("(Aucune vente)");
        } else {
            for (VenteDao.Vente v : ventes) {
                System.out.println("Vente#" + v.idVente()
                        + " | client=" + v.idClient()
                        + " | qte=" + v.quantite()
                        + " | date=" + v.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}

case 4 -> {
    // Consulter ventes par client
    try {
        int idClient = readInt("ID client: ");
        List<VenteDao.Vente> ventes = service.ventesParClient(idClient);

        if (ventes.isEmpty()) {
            System.out.println("(Aucune vente)");
        } else {
            for (VenteDao.Vente v : ventes) {
                System.out.println("Vente#" + v.idVente()
                        + " | med=" + v.idMedicament()
                        + " | qte=" + v.quantite()
                        + " | date=" + v.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}
```

```
            case 5 -> {
                // Consulter ventes par période (dates ISO)
                try {
                    System.out.println("Format ISO attendu: 2026-01-26T14:30:00");
                    String start = readLine("Date début: ");
                    String end = readLine("Date fin  : ");

                    List<VenteDao.Vente> ventes = service.ventesParPeriode(start, end);

                    if (ventes.isEmpty()) {
                        System.out.println("(Aucune vente)");
                    } else {
                        for (VenteDao.Vente v : ventes) {
                            System.out.println("Vente#" + v.idVente()
                                    + " | med=" + v.idMedicament()
                                    + " | client=" + v.idClient()
                                    + " | qte=" + v.quantite()
                                    + " | date=" + v.date());
                        }
                    }
                } catch (Exception e) {
                    printError(e);
                }
            }

            case 6 -> {
                // Créer un client
                try {
                    String nom = readLine("Nom: ");
                    String prenom = readLine("Prénom: ");
                    String email = readLine("Email: ");
                    String adresse = readLine("Adresse: ");

                    int idClient = service.creerClient(nom, prenom, email, adresse);
                    System.out.println("☑ Client créé. ID Client = " + idClient);
                } catch (Exception e) {
                    printError(e);
                }
            }

            case 0 -> {
                // Logout pharmacien
                return;
            }

            default -> System.out.println("Choix invalide.");
        }
    }
}
```

```java
// ============================================================
//                    MENU GESTIONNAIRE
// ============================================================


private static void menuGestionnaire(Session session) {
    // Service métier Gestionnaire
    GestionnaireService service = new GestionnaireService(session);

    while (true) {
        System.out.println();
        System.out.println("===== MENU GESTIONNAIRE =====");
        System.out.println("1) Ajouter médicament");
        System.out.println("2) Mettre à jour stock (valeur exacte)");
        System.out.println("3) Afficher stock critique");
        System.out.println("4) Créer commande (réapprovisionnement)");
        System.out.println("5) Consulter commandes");
        System.out.println("6) Historique stock d'un médicament");
        System.out.println("7) Lister tous les médicaments");
        System.out.println("8) Modifier prix d'un médicament");
        System.out.println("0) Logout");

        int choix = readInt("Choix: ");

        switch (choix) {

            case 1 -> {
                // Ajouter médicament : insert Medicament + log historique (+stockInitial)
                try {
                    String nom = readLine("Nom: ");
                    String dosage = readLine("Dosage: ");
                    int stock = readInt("Stock initial: ");
                    double prix = readDouble("Prix unitaire: ");

                    int id = service.ajouterMedicament(nom, dosage, stock, prix);
                    System.out.println("☑ Médicament ajouté. ID = " + id);
                } catch (Exception e) {
                    printError(e);
                }
            }

            case 2 -> {
                // Mettre à jour stock exact : calcule delta + update stock + historique
                try {
                    int idMed = readInt("ID medicament: ");
                    int stock = readInt("Nouveau stock: ");

                    service.mettreAJourStockExact(idMed, stock);
                    System.out.println("☑ Stock mis à jour.");
                } catch (Exception e) {
                    printError(e);
                }
            }
```

```java
case 3 -> {
    // Afficher stock critique (stock <= seuil)
    try {
        int seuil = readInt("Seuil critique (ex: 5): ");
        List<MedicamentDao.Medicament> list = service.stockCritique(seuil);

        if (list.isEmpty()) {
            System.out.println("(Aucun médicament en stock critique)");
        } else {
            for (MedicamentDao.Medicament m : list) {
                System.out.println("Med#" + m.id()
                        + " | " + m.nom()
                        + " | " + m.dosage()
                        + " | stock=" + m.stock()
                        + " | prix=" + m.prixUnitaire());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}

case 4 -> {
    // Créer commande (réapprovisionnement)
    try {
        int idMed = readInt("ID medicament: ");
        int qte = readInt("Quantité: ");

        int idCmd = service.creerCommande(idMed, qte);
        System.out.println("☑ Commande créée. ID = " + idCmd);
    } catch (Exception e) {
        printError(e);
    }
}

case 5 -> {
    // Consulter toutes les commandes
    try {
        List<CommandeDao.Commande> cmds = service.consulterCommandes();

        if (cmds.isEmpty()) {
            System.out.println("(Aucune commande)");
        } else {
            for (CommandeDao.Commande c : cmds) {
                System.out.println("Cmd#" + c.id()
                        + " | med=" + c.idMedicament()
                        + " | qte=" + c.quantite()
                        + " | date=" + c.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}
```

```
case 6 -> {
    // Historique stock d'un médicament
    try {
        int idMed = readInt("ID medicament: ");
        List<StockHistoriqueDao.StockHist> hist = service.historiqueStock(idMed);

        if (hist.isEmpty()) {
            System.out.println("(Historique vide)");
        } else {
            for (StockHistoriqueDao.StockHist h : hist) {
                System.out.println("Hist#" + h.id()
                        + " | med=" + h.idMedicament()
                        + " | delta=" + h.quantite()
                        + " | date=" + h.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}

case 7 -> {
    // Lister tous les médicaments
    try {
        List<MedicamentDao.Medicament> meds = service.listMedicaments();

        if (meds.isEmpty()) {
            System.out.println("(Aucun médicament)");
        } else {
            for (MedicamentDao.Medicament m : meds) {
                System.out.println("Med#" + m.id()
                        + " | " + m.nom()
                        + " | " + m.dosage()
                        + " | stock=" + m.stock()
                        + " | prix=" + m.prixUnitaire());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}

case 8 -> {
    // Modifier prix d'un médicament
    try {
        int idMed = readInt("ID medicament: ");
        double prix = readDouble("Nouveau prix: ");

        service.modifierPrix(idMed, prix);
        System.out.println("☑ Prix modifié.");
    } catch (Exception e) {
        printError(e);
    }
}
```

```java
    // ============================================================
    //                        MÉTHODES UTILES
    // ============================================================


     // Lecture d'une chaîne de caractères (String) dans la console.

    private static String readLine(String msg) {
        System.out.print(msg);
        return sc.nextLine().trim();
    }


     //Lecture sécurisée d'un entier dans la console (boucle tant que saisie invalide).

    private static int readInt(String msg) {
        while (true) {
            System.out.print(msg);
            String s = sc.nextLine().trim();
            try {
                return Integer.parseInt(s);
            } catch (Exception ignored) {
                System.out.println("Entrez un nombre valide.");
            }
        }
    }


     // Lecture sécurisée d'un double dans la console.

    private static double readDouble(String msg) {
        while (true) {
            System.out.print(msg);
            String s = sc.nextLine().trim().replace(",", ".");
            try {
                return Double.parseDouble(s);
            } catch (Exception ignored) {
                System.out.println("Entrez un nombre (double) valide.");
            }
        }
    }


     // Affichage standardisé des erreurs côté console.

    private static void printError(Exception e) {
        System.out.println("❌ Erreur: " + e.getMessage());
    }
}
```

Execution (exemples):

Enregistrer une vente (Pharmacien) :

```
----- LOGIN -----
Login: pharma
Password: pharma123
☑ Connecté: PHARMACIEN | Pharma Doe

===== MENU PHARMACIEN =====
1) Enregistrer une vente
2) Annuler une vente
3) Consulter ventes par médicament
4) Consulter ventes par client
5) Consulter ventes par période
6) Créer un client
0) Logout
Choix: 1
ID client: 1
ID medicament: 2
Quantité: 1
☑ Vente enregistrée. ID Vente = 2
```

Annulation d'une vente (Pharmacien) :

```
----- LOGIN -----
Login: pharma
Password: pharma123
☑ Connecté: PHARMACIEN | Pharma Doe

===== MENU PHARMACIEN =====
1) Enregistrer une vente
2) Annuler une vente
3) Consulter ventes par médicament
4) Consulter ventes par client
5) Consulter ventes par période
6) Créer un client
0) Logout
Choix: 2
ID vente à annuler: 1
☑ Vente annulée.
```

Ajouter médécament (Gestionnaire):

```
----- LOGIN -----
Login: gest
Password: gest123
☑ Connecté: GESTIONNAIRE | Gestion Doe

===== MENU GESTIONNAIRE =====
1) Ajouter médicament
2) Mettre à jour stock (valeur exacte)
3) Afficher stock critique
4) Créer commande (réapprovisionnement)
5) Consulter commandes
6) Historique stock d'un médicament
7) Lister tous les médicaments
8) Modifier prix d'un médicament
0) Logout
Choix: 1
Nom: Motilium
Dosage: 10
Stock initial: 14
Prix unitaire: 15
☑ Médicament ajouté. ID = 7
```

Modifier prix d'un mediament (Gestionnaire) :

```
----- LOGIN -----
Login: gest
Password: gest123
☑ Connecté: GESTIONNAIRE | Gestion Doe

===== MENU GESTIONNAIRE =====
1) Ajouter médicament
2) Mettre à jour stock (valeur exacte)
3) Afficher stock critique
4) Créer commande (réapprovisionnement)
5) Consulter commandes
6) Historique stock d'un médicament
7) Lister tous les médicaments
8) Modifier prix d'un médicament
0) Logout
Choix: 8
ID medicament: 7
Nouveau prix: 13
☑ Prix modifié.
```

Conclusion:

En conclusion, ce projet a permis de réaliser une application complète de gestion d'une pharmacie en Java avec une base de données SQLite, en utilisant uniquement JDBC. L'application répond aux besoins principaux : authentification des utilisateurs (pharmacien/gestionnaire), gestion des médicaments et du stock, enregistrement et consultation des ventes, gestion des commandes de réapprovisionnement, ainsi que le suivi du stock critique et de l'historique des mouvements de stock.

Ce travail a également permis de mettre en pratique une architecture en couches (DAO/Service/App), ce qui rend le code plus clair, mieux organisé et plus facile à maintenir. Des améliorations futures peuvent inclure une interface graphique plus avancée, des rapports plus détaillés et une gestion plus complète des notifications.